

# 操作系统

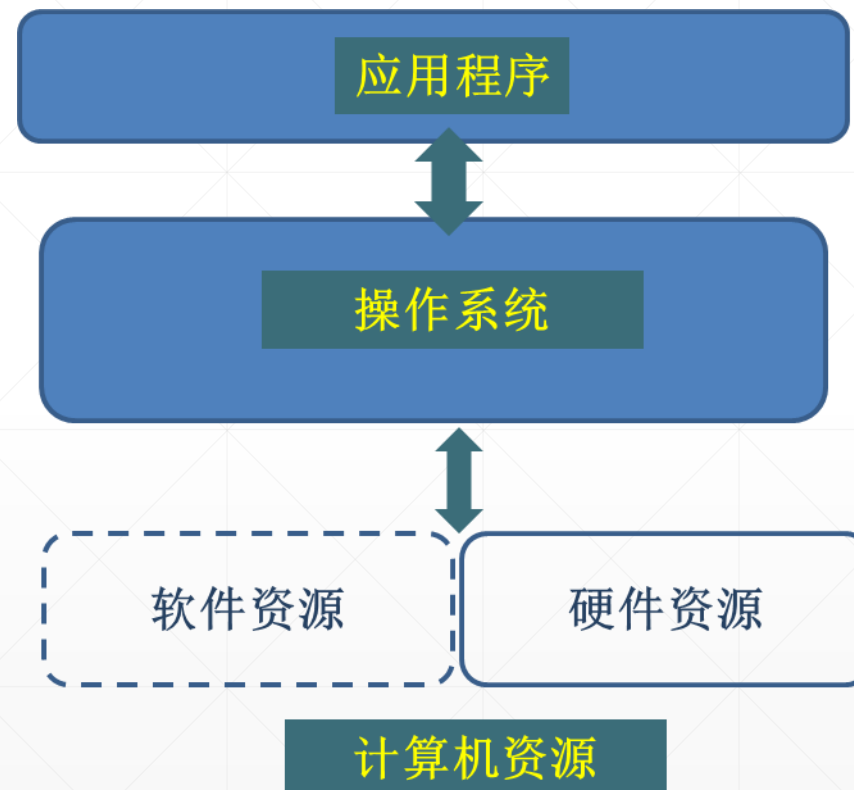
## 第一章 绪论

---

同济大学计算机系

# 操作系统定义

- 操作系统 (Operating System) 是最重要的系统软件，介于其它软件（应用程序）和 计算机硬件之间，负责管理计算机系统内的一切**资源**，为所有应用**程序**提供资源访问服务。



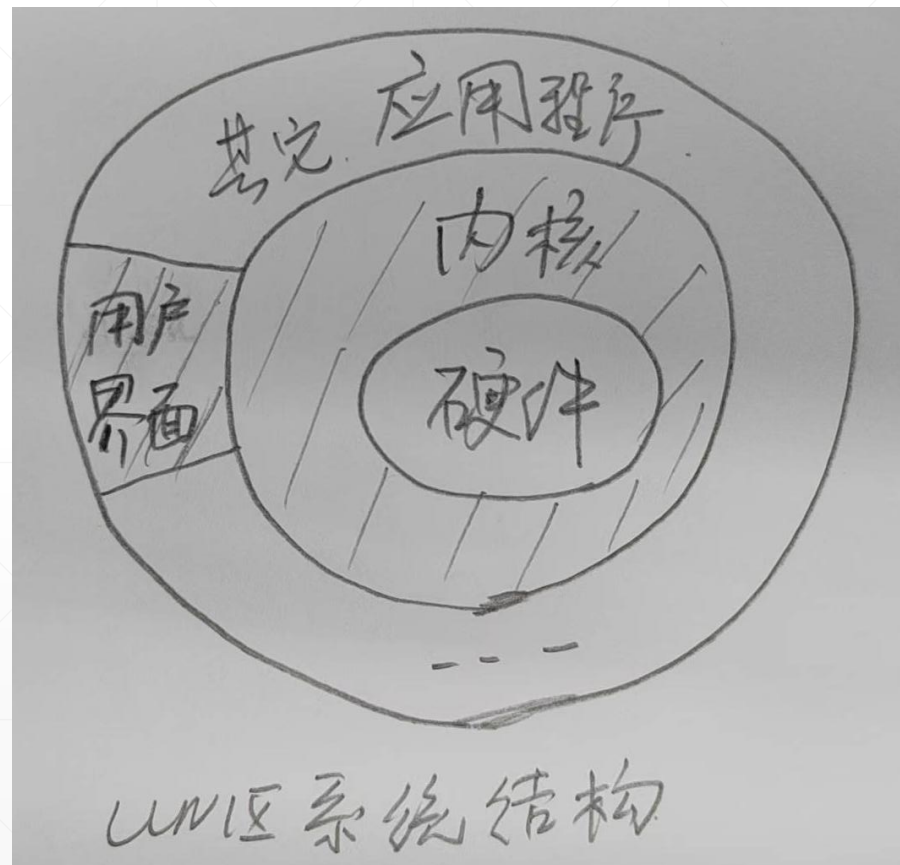


# 操作系统核心价值：为应用程序提供 算力 和 数据

- 算力是执行的可能
- 数据是程序处理的信息，来自 键盘、磁盘、本机和远程运行的其它程序

# 操作系统结构

- 为每台计算机配备的操作系统包括，用户界面 和 内核。这两个程序彼此独立。
- 用户界面是人机接口，负责接纳用户提交的作业，反馈作业的运行结果；控制作业的执行过程。是一个应用程序。
- 内核管理、调度计算机资源，执行系统接纳的作业（一组应用程序）。包括4个功能模块，分别用来管理CPU、内存、外设和存放在磁盘上的数据。



# 操作系统的5大功能模块

用户界面 (User Interface)

Unix / Linux, shell  
Windows是 Powershell 或 Cmd

进程  
管理  
(CPU)

内存  
管理  
(MM)

外设  
管理  
(IO)

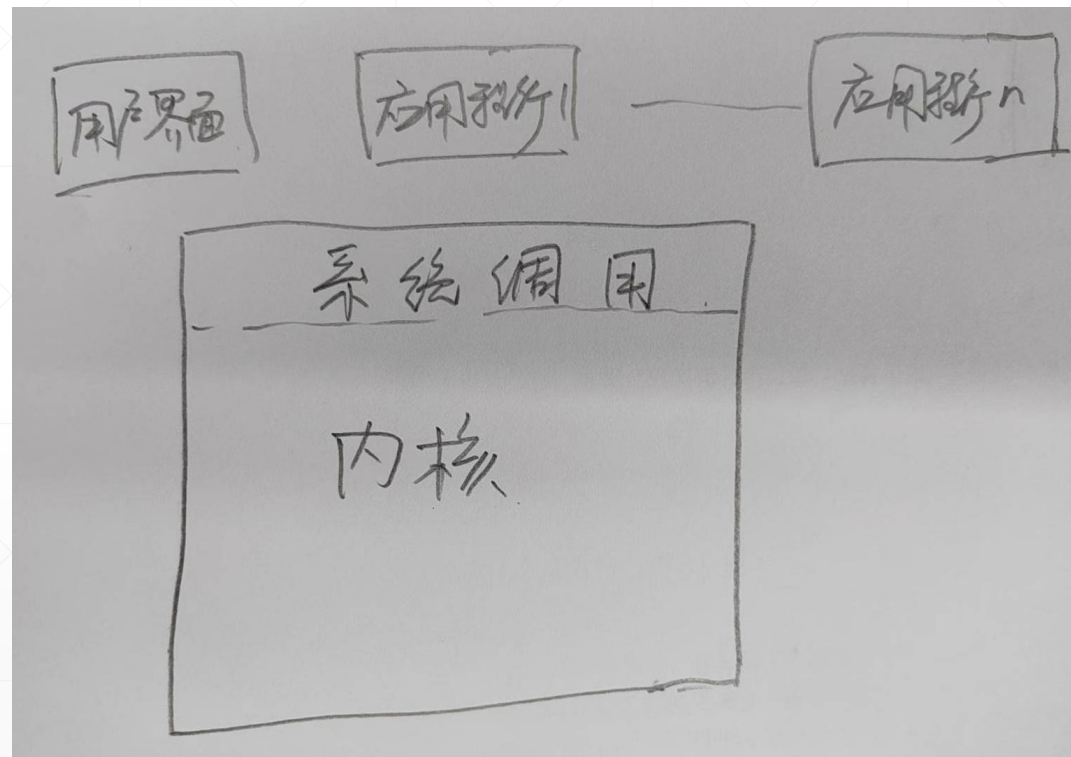
文件  
管理  
(On  
Disk  
Data)

内核包括4个功能模块，分别用来管理CPU、内存、外设和存放在磁盘上的数据。

# 系统调用是内核和应用程序的接口

需要访问系统资源时，应用程序执行系统调用向内核发请求。

收到请求后，内核执行系统调用对应的子函数，为用户提供系统资源访问、控制服务。





# Part 1 深入理解 应用程序 和 计算机系统

- 程序员视角

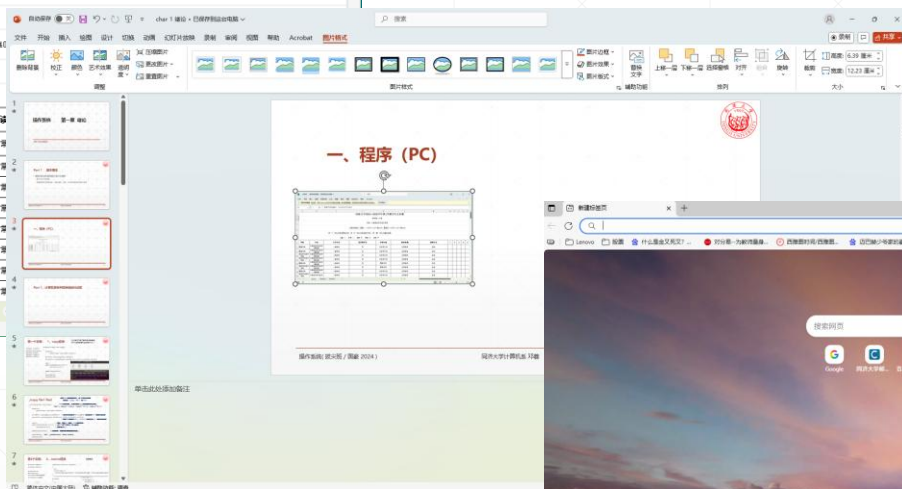


# 一、应用程序

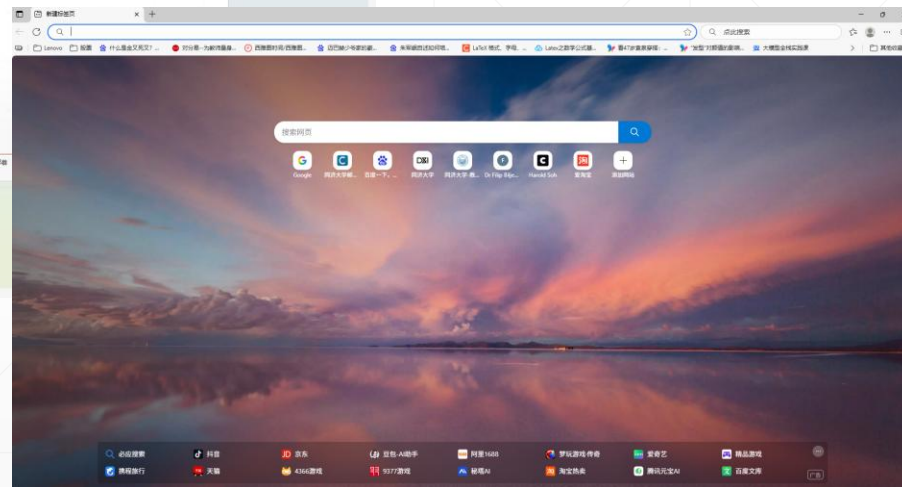
Excle程序



PowerPointer程序



Edge程序



# PC机上运行的GUI（Graphical User Interface，图形用户界面）程序

每个程序一个图标，一个窗口。双击启动，用鼠标、键盘工作

```
Windows PowerShell
版权所有 (C) Microsoft Corporation。保留所有权利。

安装最新的 PowerShell, 了解新功能和改进! https://aka.ms/PSWindows

PS C:\Users\Pretty-Girl> ping 127.1.1.1

正在 Ping 127.1.1.1 具有 32 字节的数据:
来自 127.1.1.1 的回复: 字节=32 时间<1ms TTL=128
来自 127.1.1.1 的回复: 字节=32 时间<1ms TTL=128
来自 127.1.1.1 的回复: 字节=32 时间<1ms TTL=128
来自 127.1.1.1 的回复: 字节=32 时间<1ms TTL=128

127.1.1.1 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
往返行程的估计时间(以毫秒为单位):
    最短 = 0ms, 最长 = 0ms, 平均 = 0ms
PS C:\Users\Pretty-Girl> date

2025年9月14日 12:07:30

PS C:\Users\Pretty-Girl> echo hello OS
hello
OS
PS C:\Users\Pretty-Girl> |
```

CLI程序, ping

CLI程序, date

CLI程序, echo

高程作业

.....

# 服务器、PC机上运行的CLI(Command Line Interface, 命令行界面) 程序

没有图标, 没有窗口。回车启动, 键盘工作

# 应用程序示例 CLI程序copy

```
int main( int argc, char* argv[] )
{
    if(argc!=3)
        printf("Usage: copy oldfile newfile\n");

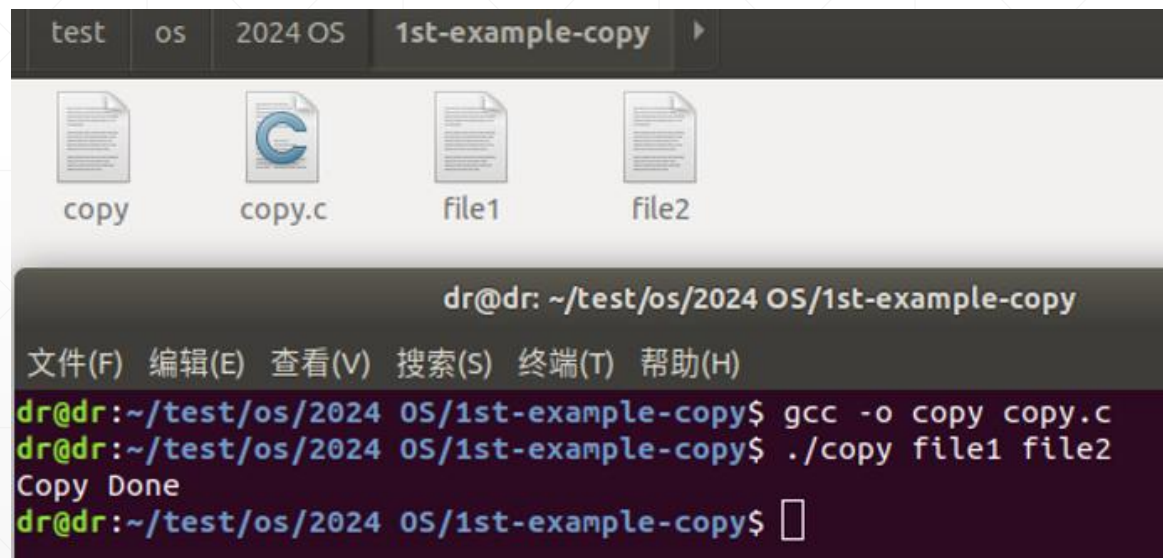
    int oldFile = open(argv[1],O_RDONLY);
    int newFile = open(argv[2],O_WRONLY|O_CREAT,S_IRUSR|S_IWUSR);

    char c;
    while( read(oldFile,&c,1) == 1 )
        write(newFile,&c,1);

    printf("Copy Done\n");

    close(oldFile);
    close(newFile);
}
```

```
#include <unistd.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdlib.h>
#include <stdio.h>
```



```
test os 2024 OS 1st-example-copy
copy copy.c file1 file2
dr@dr: ~/test/os/2024 OS/1st-example-copy
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
dr@dr:~/test/os/2024 OS/1st-example-copy$ gcc -o copy copy.c
dr@dr:~/test/os/2024 OS/1st-example-copy$ ./copy file1 file2
Copy Done
dr@dr:~/test/os/2024 OS/1st-example-copy$
```

copy程序一次一字节地将已有文件file1的内容复制到新文件file2



# 应用程序的组成：包括 代码 和 数据

```
int main( int argc, char* argv[] )
{
    if(argc!=3)
        printf("Usage: copy oldfile newfile\n");

    int oldFile = open(argv[1],O_RDONLY);
    int newFile = open(argv[2],O_WRONLY|O_CREAT,S_IRUSR|S_IWUSR);

    char c;
    while( read(oldFile,&c,1) == 1 )
        write(newFile,&c,1);

    printf("Copy Done\n");

    close(oldFile);
    close(newFile);
}
```

## copy程序

代码：函数 main, printf, open, read, write, close  
数据：所有的常量（金色） 和 变量（红色）

备注1：main是程序员写的函数，其余是系统提供的库函数

备注2：常量，只读数据

局部变量，定义在函数内部的变量

全局变量，定义在函数外部（copy程序没有）

# 命令行参数, argc、argv

```
int main( int argc, char* argv[] )
{
    if(argc!=3)
        printf("Usage: copy oldfile newfile\n");

    int oldFile = open(argv[1],O_RDONLY);
    int newFile = open(argv[2],O_WRONLY|O_CREAT,S_IRUSR|S_IWUSR);

    char c;
    while( read(oldFile,&c,1) == 1 )
        write(newFile,&c,1);

    printf("Copy Done\n");

    close(oldFile);
    close(newFile);
}
```

- 命令行参数 是用户在命令行中为应用程序提供的参数，是main函数的入口参数。argv，命令行参数数组； argc是命令行参数的个数。

例： ./copy file1 file2

执行copy程序的命令行，有3个命令行参数。

∴ argc = 3, argv数组有4个元素

argv[0]= "./copy\0" , argv[1]= "file1\0" ,  
argv[2]= "file2\0" , argv[3]= "\0" ,



## 附录：copy程序源代码注释

```
int main( int argc, char* argv[] )
{
    if(argc!=3)
        printf("Usage: copy oldfile newfile\n");

    int oldFile = open(argv[1],O_RDONLY); // 读打开已有文件 file1, oldFile 是用来读 file1 的文件描述符
    int newFile = open(argv[2],O_WRONLY|O_CREAT,S_IRUSR|S_IWUSR);
                                   // 创建一个新文件 file2, newFile 是用来写 file2 的文件描述符

    char c;                        // 循环, 直至file1读完, read函数返回0
    while( read(oldFile,&c,1) == 1 ) // 从file1读一个字节, 存入变量c
        write(newFile,&c,1);        // 将其写入file2

    printf("Copy Done\n"); // 人机交互。写屏幕告知程序运行结束。

    close(oldFile); // 文件, 访问完及时关闭。这是一个好习惯。
    close(newFile);
}
```



# 作业

- 改写copy程序，将文件file1的内容复制到文件file2，每次复制4096字节。
- 比较改写之前 和 之后的运行速度。初步解释观察到的现象。

## ■ 阅读、编写方阵自乘程序，回答以下问题

```
# include <stdlib.h>
# include <stdio.h>

# define M 16
int version = 1;
int matrixOriginal[M][M];
int matrixDes [M][M];
```

```
void produce ( int row, int column )
```

```
{
```

```
    int i;
```

```
    for (i=0;i<M;i++)
```

```
        matrixDes[row][column] += matrixOriginal[row][i] * matrixOriginal[i][column];
```

```
}
```

```
int main()
```

```
{
```

```
    int i , j;
```

```
    for (i=0;i<M;i++)
```

```
        for (j=0;j<M;j++) {
```

```
            matrixOriginal[i][j] = 1;
```

```
            matrixDes[i][j] = 0;
```

```
        }
```

```
        for (i=0;i<M;i++)
```

```
            for (j=0;j<M;j++)
```

```
                produce(i , j);
```

```
        printf("matrixDes == \n");
```

```
        for (i=0;i<M;i++) {
```

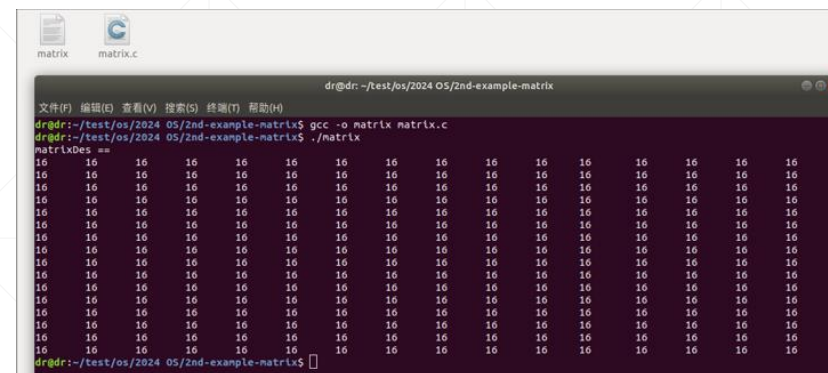
```
            for (j=0;j<M;j++)
```

```
                printf("%d\t", matrixDes[i][j]);
```

```
                printf("\n");
```

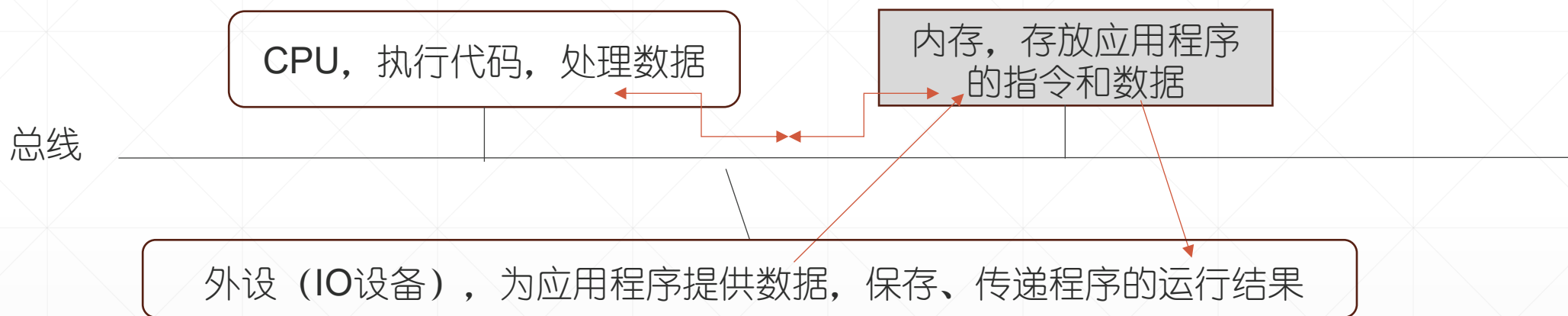
```
        }
```

```
}
```



1、matrix程序的代码，包括？  
2、指出matrix程序的只读数据，全局变量和局部变量。  
3、M是什么（是编译器使用的符号，不是应用程序的数据）？  
请将M修改为4096，测速。为什么程序慢那么多？加速它。

## 二、资源的使用



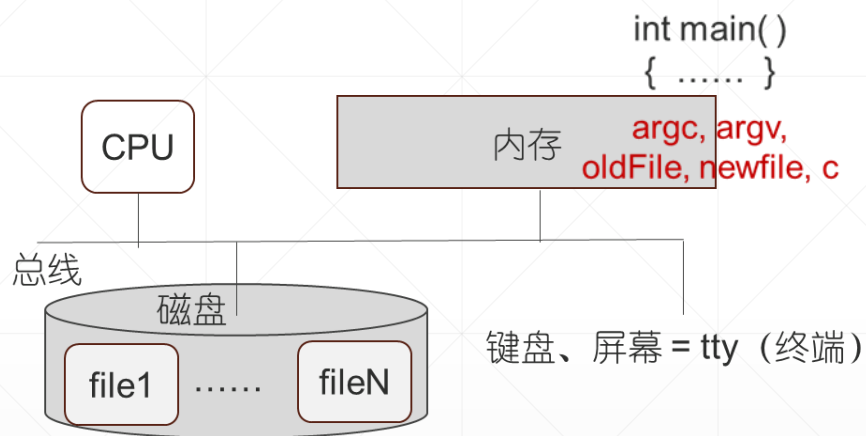
图、冯诺依曼计算机系统结构（基于存储程序思想）

# 细节



# 程序对资源的使用：内存

执行中的应用程序使用内存资源，存放代码和正在处理的数据。

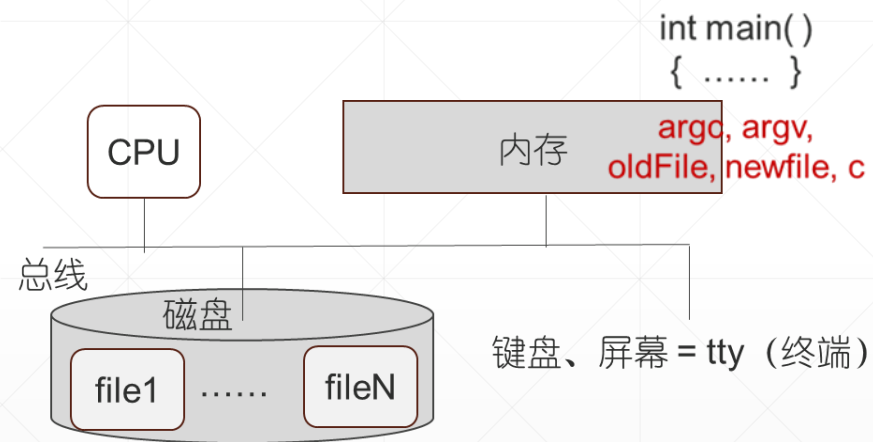


备注：未执行的程序，是磁盘上存放的可执行文件，不占内存资源。

双击、回车启动应用程序时，系统为其代码、数据分配内存空间，随后从磁盘载入代码、数据。

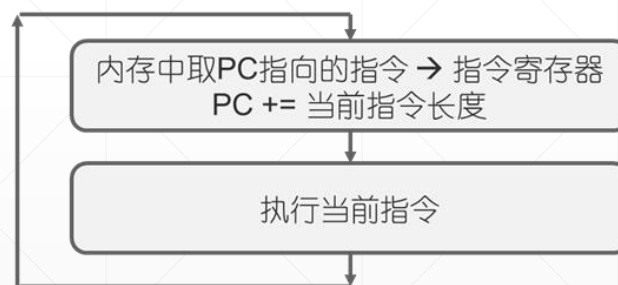
# 程序对资源的使用：CPU

## 顺序执行内存中的指令

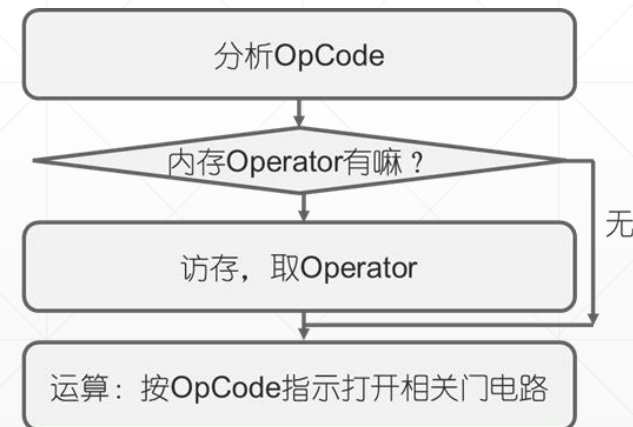


## CPU执行指令的硬件逻辑

设置程序计数器 PC，存放下条指令的地址。  
当前指令执行完毕后，访存读取、执行PC所指下条指令，同时修改  $PC += \text{下条指令长度}$ ，指向再下条指令。循环。。。



计算机指令：OpCode Operator1 Operator2



双击、回车启动应用程序时， `main()` 函数第一条指令的地址  $\rightarrow$  PC。CPU自动执行程序，直至运行结束。

# 程序对资源的使用：磁盘、屏幕 和 软件资源

```
int main( int argc, char* argv[] )
{
    if(argc!=3)
        printf("Usage: copy oldfile newfile\n");

    int oldFile = open(argv[1],O_RDONLY);
    int newFile = open(argv[2],O_WRONLY|O_CREAT,S_IRUSR|S_IWUSR);

    char c;
    while( read(oldFile,&c,1) == 1 )
        write(newFile,&c,1);

    printf("Copy Done\n");

    close(oldFile);
    close(newFile);
}
```

./copy file1 file2

